

# 程设大作业作业报告

## ——程设大富翁

72 组 陈侯超 冷子涵 张城玮

### 一、程序功能介绍

《程设大富翁》基于 Qt 开发，是一款以大富翁游戏的形式，帮助用户学习和巩固 c++面向对象编程相关知识的元学习产品。

#### • 主界面

主界面主要由五部分组成：左侧的大矩形区域，呈现棋盘；右上角为状态栏，显示当前回合行动的玩家、每名玩家剩余资产、持有道具数等信息；状态栏下方为骰子区，在这一区域掷骰子和显示骰子点数；骰子区下方为日志栏，游戏事件将显示在这里。最下方显示每名玩家持有的全部地产和道具信息。如果场上玩家持有的地产和道具总数太多，无法一次性全部显示，会显示一部分，玩家可以通过操作滚动条看到其余部分。左上角为游戏菜单，包含开始新游戏、开启 DEBUG 模式、退出三个选项。可以设置全屏模式，此时界面各部分的尺寸会相应调整，以匹配新的窗口尺寸。

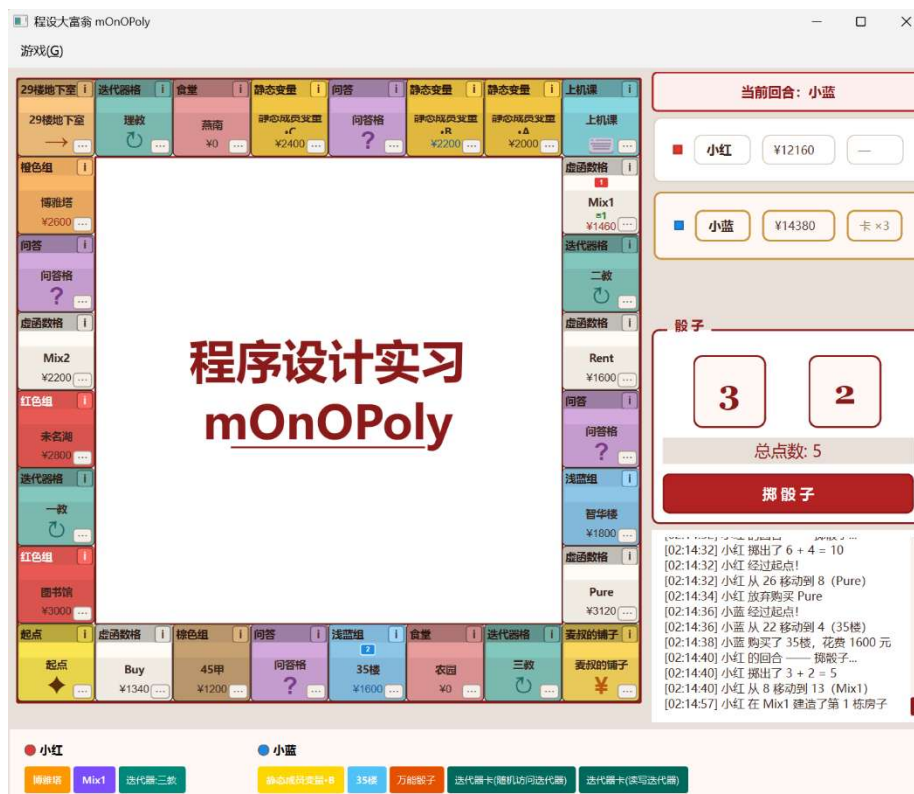


图 1 游戏中主界面

• 开始新游戏

未开始游戏时点击屏幕中央出现的“开始游戏”按钮，或是从菜单中选择开始新游戏，都可以打开新游戏设置窗口。在这个页面可以选择玩家总数，以及每个玩家是人类还是 AI。AI 通过一定的规则判断进行决策。AI 的具体行为逻辑将在后文对应模块具体介绍。

可以分别设定每个 AI 的难度，不同难度对应 AI 不同的初始资金数额。

可以为每名玩家使用默认名字，也可以自己指定名字。



图 2-1 新游戏设置界面



图 2-2 设定 AI 难度

• 地块

地块是棋盘的基本单位。一个地块包含如下元素：左上角的文字指出该地块的类型；单击右上角的“i”图标，可以查看地块信息；单击右下角的“...”图标，可以查看地块详细信息；最中间的文字指出该地块的名称。形如图 3-1 中红色“1”图标的是玩家图标，它表示编号对应的玩家正处在该地块上。



图 3-1 一个地块

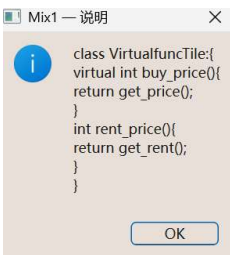


图 3-2 查看地块信息



图 3-3 查看地块详细信息

棋盘左下角的地块为起点。经过起点可以获得 5000 资金，若正好停留在起点，可以获得 10000 资金。



图 3-4 起点

[17:52:19] 小蓝 经过起点!  
[17:52:19] 小蓝 从 18 移动到 0 (起点)  
[17:52:20] 小蓝 停在起点, 获得奖金 10000 元!

图 3-5 停在起点，获得资金

左上角显示的类别为各种颜色的地块是普通地产地块。当玩家前进到这类地块，若地块没有所有者，可以花费资金购买这个地块。地块最下方的数字是它的购买价格，地块没有所有者时，这个数字显示为黑色，否则为所有者的颜色。若地块的所有者为玩家自己，玩家可以花费资金建造房子。地块中下方的绿色数字表示该地块上已经建造的房子数目。若资金不足，则无法购买地块或是建造房子。若地块的所有者为其他玩家，玩家会被收租，扣除一定量资金，地块的所有者获得等额资金。若玩家剩余资金不够收租，会被判定为破产出局，之后不能再行动。AI 的策略较为保守，当 AI 剩余资产达到购买地产所需资产的两倍，就会购买资产；达到建房所需资产的三倍，就会建房。



图 3-6 普通地产地块



图 3-7 购买地产

[17:58:39] 小蓝 从 22 移动到 25 (未名湖)  
[17:58:39] 小蓝 停在 未名湖 (属于小红), 支付租金 240 元

图 3-8 收租



图 3-9 建房

[18:10:49] 小蓝 从 5 移动到 13 (Mix1)  
[18:10:50] 小蓝 资金不足, 无法升级 Mix1

图 3-10 资金不足, 无法建房

左上角显示的类别为“虚函数格”的地块是虚函数地产地块。与普通地块不同的是，这类地块包含基类购买方法、收租方法和派生类购买方法、收租方法，基类和派生类的方法行为不同。例如，如图 3-2 和图 3-3 所示，通过阅读代码，可以发现，调用虚函数地块 Mix1 的派生类收租方法可以收到原来 1.6 倍的租金。购买地块时玩家可使用道具“虚函数卡”决定执行基类的购买方法还是派生类的购买方法。收租时地块所有者可使用道具“虚函数卡”决定执行基类的收租方法还是派生类的收租方法。AI 默认不使用虚函数卡。

虚函数格有不同类型，只影响购买的 Buy，只影响收租的 Rent，二者皆有的 Mix 和纯虚函数 Pure，必须使用虚函数卡才能对 Pure 收租，否则租金为 0。



图 3-11 询问是否使用虚函数卡

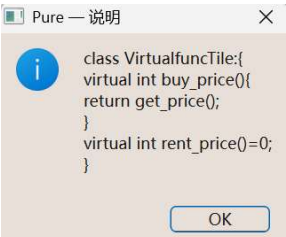


图 3-12 纯虚函数 Pure

左上角显示的类别为“食堂”的地块，取代了传统大富翁中的税收块。玩家前进到食堂块时，会消费一定金额，但如果剩余资金不够消费，就不消费。



图 3-13 食堂地块

[19:29:54] 小红 从 8 移动到 19 (燕南)  
[19:29:54] 小红 花掉了3000 元, 吃成了大卫戴!

图 3-14 在食堂消费

左上角显示的类别为“迭代器格”的地块是迭代器块。可以使用道具“迭代器卡”，按照规则在不同迭代器块之间传送。读写迭代器只能执行“++”操作，按逆时针顺序前往下一个迭代器格；双向迭代器可以执行“++”和“--”操作，前往相邻的迭代器格；随机访问迭代器可以前往任意一个迭代器格。如果不按照迭代器的规则行动，会消耗掉迭代器卡，但不能传送。AI 不会使用迭代器卡。



图 3-15 迭代器格

[15:45:17] 小红 使用迭代器卡(双向迭代器)  
-- → 传送到 三教!

图 3-16 成功使用迭代器卡

[16:20:56] 小蓝 使用了迭代器卡(读写迭代器)的 +=2 操作, 但此操作不被支持! 卡片被消耗, 无效果。

图 3-17 未按规则，使用迭代器卡失败

棋盘右下角的“麦叔的铺子”是商店，玩家每次前进到该地块，都可以消费对应金额购买一件道具。棋盘左上角的“29 楼地下室”是商店入口，玩家前进到该地块时，可以选择是否直接移动到商店。商店中每次刷新“再丢一次骰子”、“万能骰子”、“虚函数卡”、“跳过卡”和随机一种迭代器卡。AI 只有在剩余金额大于 3000 时才会从商店入口传送到商店。AI 只会买“再丢一次骰子”、“万能骰子”和“跳过卡”。



图 3-18 商店



图 3-19 商店入口

在地图上前进到问答格或上机课格，会触发问答事件。问答和上机课的题目取自历年程设考试选择题，考察面向对象编程、STL 等知识点。答对问答格

的问答，有概率获得一张随机效果卡，而答对上机课的问答，一定能获得一张随机效果卡。AI 触发问答事件时，会随机选择一个选项，如果答对，也能正常触发奖励。

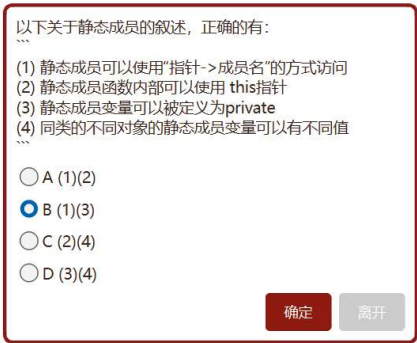


图 3-20 问答界面

[07:10:59] 小红 回答正确!  
[07:10:59] 小红 获得效果卡：再丢一次骰子!

图 3-21 回答问答正确，并获得随机效果卡

[07:07:28] 小红 进入上机课（计算机实验课），需回答一道题，下回合跳过！  
[07:07:47] 小红 回答错误。正确答案是 C。下回合跳过。

图 3-22 回答上机课错误，并跳过下一个回合

静态成员变量 A、B、C 三格都是静态成员变量地产。静态成员变量格包含静态变量 count，租金计算与 count 有关，即集齐 3 个静态成员变量地产可以获得大量额外租金。



图 3-23 静态变量格

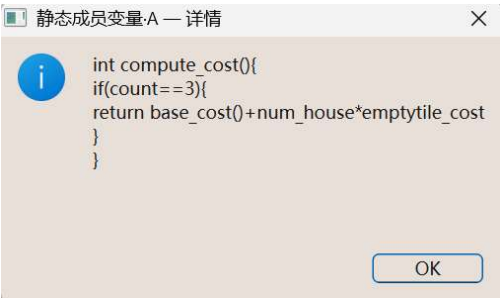


图 3-24 对静态成员变量格的介绍

### • 掷骰子

每回合开始时，当前回合行动的玩家掷骰子。一次掷两枚骰子，每枚骰子的点数在 1 至 6 之间。该玩家在地图上沿逆时针方向前进与投出点数之和相等的格数。如果掷出对子，即两枚骰子的点数相同，当前回合结束后，该玩家可以再行动一个回合。



图 4-1 掷骰子



图 4-2 掷出对子

掷骰子前，如果有道具“万能骰子”，可选择使用，由玩家指定两枚骰子的点数。掷完骰子后，如果有道具“再掷一次骰子”，可选择使用，当前掷出的点



数不算，重新掷骰子。AI 如果有这两种道具，一定会使用，其中使用万能骰子时会指定点数为 6+6。

• **DEBUG 模式**

从左上角菜单可以选择进入 DEBUG 模式。该模式下，玩家可以控制骰子的点数，也可以选择是否触发知识点事件。

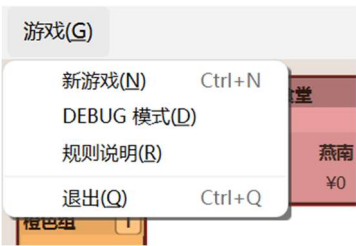


图 5-1 进入 DEBUG 模式



图 5-2 手动设置骰子点数



图 5-3 选择是否触发随机事件

• **知识点事件**

每个人类玩家的回合结束时，有 1/10 的概率触发知识点事件，随机讲解一个程设知识点，并奖励一张随机效果卡。



图 6-1 知识点界面

[19:57:10] 小蓝 触发了知识点事件!  
[20:01:21] 小蓝 完成阅读，获得效果卡：万能骰子!

图 6-2 完成阅读，获得随机效果卡

• **道具/效果卡**

道具包括再掷一次骰子、万能骰子、跳过卡、虚函数卡和迭代器卡。再掷一次骰子在掷出骰子后选择是否使用。万能骰子在掷出骰子前选择是否使用，并指定两枚骰子的点数。跳过卡在遇到的地块有负面收益时选择是否使用，可以跳过被收租、食堂消费。在虚函数地块上购买地产、建房和收租时，可以使用虚函数卡，选择不执行对应的基类函数，而执行派生类函数。迭代器卡包含读写迭代器、双向迭代器和随机访问迭代器，用于在迭代器格之间传送。

获取道具的方式包括：在商店消耗资金购买、触发知识点事件直接获得、问答格答对题目概率获得、上机课答对题目必定获得。

## 二、项目各模块与类设计细节

### • game.h 与 game.cpp

Game 是整个《程设大富翁》项目的核心。在本项目中，所有需要用到的参数和方法定义在头文件中，具体实现在同名 cpp 文件中，这一模式之后不再重复叙述。Game 将游戏流程中的所有可能事件定义成了方法，主要包括初始化、掷骰子、玩家移动、玩家抵达目标位置、玩家购买地产、玩家建房、玩家使用道具、回合结束判定、胜利判定、日志打印；触发事件时，只需要调用相应方法。Game 还维护了指向棋盘对象和玩家对象的指针，用于更新棋盘和玩家的状态。具体而言：

初始化部分，接收玩家数目和每个玩家的名字、编号、是否为 AI 及 AI 难度信息，创建玩家类；创建棋盘类。将当前玩家、骰子点数等全部设为默认。

掷骰子部分，先判断玩家是否因为上机课被跳过了回合，这是通过 player 类的内置方法实现的；若没有被跳过，通过 player 类的内置方法检测持有道具，若有万能骰子，询问是否使用万能骰子。若使用万能骰子，则发出“万能骰子”信号，由玩家指定两个骰子的点数后，传入方法 onDiceRolled()。否则通过指针操作骰子类的实例，掷出点数。掷完骰子后，再检测道具，若有“再丢一次骰子”，询问是否使用。若使用，则重复上述流程，否则保存掷出的点数和是否掷出对子的信息，调用玩家移动方法。

玩家移动部分，通过维护指向当前回合玩家的指针，改变玩家的坐标。执行途经地块的 passBy() 方法，这是 tile 类的内置方法。途经地块的 passBy() 方法全部调用完后，调用玩家抵达目标位置方法。

玩家抵达目标位置部分，调用 board 棋盘类的内置方法，根据玩家的坐标，返回指向玩家所在地块的指针。根据所在地块的类型决定后续操作。若为其他玩家地产、食堂这类可以使用跳过卡的地块，检测玩家是否持有跳过卡，如果有，再执行是否使用跳过卡的判定。若使用跳过卡，直接跳到回合结束判定，否则执行地块的 landOn() 方法。这也是 tile 类的内置方法。迭代器卡的使用也基本类似，只有在迭代器块上会增加使用迭代器卡判定流程。

回合结束判定部分，如果先前掷出对子，不更新当前玩家指针，开始新回合；否则，通过 nextPlayer() 方法，让当前玩家指针指向下一个没有破产的玩家。玩家是否破产是由 player 类的内置方法 isBankrupt() 进行判定的。如果是人类玩家的回合，还会增加触发知识点事件判定。

玩家购买地产方法在玩家到达地产地块时调用，会有一步判定，判定玩家是否到达的是一个能购买的地块。通过 player 类的内置方法 canAfford() 判定是否能够支付，若不能支付，直接调用回合结束判定，玩家的当前回合不会再有操作了。否则，通过 player 类的内置方法 payMoney 扣款，并调用

PropertyTile 类的 setOwner() 和 player 类的 addProperty() 方法，建立玩家和地块的从属关系。玩家响应后，同样会调用回合结束判定。

玩家建房方法完全类似，同样是维护指向当前玩家对象的指针和指向玩家所处地块对象的指针，先判定能否支付，再根据玩家响应更新玩家和地块状态，最后调用回合结束判定。

问答与上机课部分，传入一个 question 类实例作为参数，接收玩家的答案，与传入实例的正确答案参数比较，判断玩家是否回答正确；若获得了奖励，则创建 card 对象，执行 player 类的内置方法 addEffectCard()，将创建的 card 对象增加到当前玩家的道具类数组中。最后调用回合结束判定。

在商店购买道具同理，创建 card 对象，由 addEffectCard() 将创建的对象“添加”到玩家的持有道具中。购买完成后调用回合结束判定。

每次判定前都有判断当前玩家的玩家类型是否为 AI 这一流程，如果是 AI 玩家，则根据一定的规则判断，给出信号，而不是等待玩家交互给出信号。

以上所有流程都连接着 eventlog，实时更新日志。

#### • player.h 与 player.cpp

定义了玩家类。一盘游戏中的每名玩家都是一个玩家类的实例。玩家类包含了大量方法，根据功能，大体划分为：

属性访问方法。如 id() 返回玩家编号、money() 返回玩家资产、position() 返回玩家坐标、isBankrupt() 返回玩家是否破产等。

金钱操作方法。包含收到金钱的 receiveMoney() 方法、给出金钱的 payMoney() 方法、给其他玩家金钱的 payMoneyTo() 方法、判断是否能够支付的 canAfford() 方法。receiveMoney() 和 canAfford() 两个方法都只包括金额这一个参数；payMoney() 和 payMoneyTo() 都要传入当前这局游戏的 Game 对象的指针，以落实产生的影响；payMoneyTo() 还需要传入其他玩家的指针。

移动方法。它接收坐标，修改玩家的坐标，并统计中途经过的坐标。

跳过回合方法。它用于设置“跳过下一回合”状态。

效果卡管理方法。它维护一个 card 类的可变数组，并有添加效果卡的 addEffectCard() 方法和使用效果卡的 useEffectCard() 方法，还有 hasEffectCard()，接收 card 类型参数，返回是否持有这类效果卡。

地产管理方法。它维护一个 tile 类的可变数组，并有添加地产的 addProperty() 方法等。

重置方法，用于还原为默认值。



- **board.h 与 board.cpp**

定义了棋盘类。它维护一个 `tile` 类的可变数组，按顺序将全部地块的指针添加进去；针对特殊的迭代器块，单独又建立一个迭代器派生类的可变数组，将全部迭代器块的指针添加进去。它的主要作用是根据坐标，返回指向对应地块的指针，充当外界和 `tile` 类的“中转站”。

- **tile.h 与 tile.cpp**

定义了地块基类。地块类也包含了大量方法，根据功能，大体划分为：

属性访问方法。如 `index()` 返回地块编号，`group()` 返回所属颜色组，`titleDetail()` 返回详细信息等，

`landOn()` 方法，在玩家到达地块时调用。和 `player` 的移动方法配合使用。

`passBy()` 方法，在玩家经过地块时调用。和 `player` 的移动方法配合使用。

在地块基类的基础上，定义了各种地块的派生类，有：

起点块 `StartTile` 类。修改 `landOn()` 和 `passBy()`，用于实现起点奖金。

可购买地产 `PropertyTile` 类。它的特有参数包括 `owner`，这是一个指向所有者玩家的指针；特有方法包括用于建房的 `buildHouse()`、统计房子数目的 `houses()`、根据房子数目计算租金的 `calculateRent()` 等。

静态成员变量地产 `StaticvalTile` 类，它又是 `PropertyTile` 的派生类。它有一个 `ownsFullGroup()` 方法，用于判定玩家是否集齐了全部 3 个静态成员变量地产，以实现“集齐静态变量地产，获得额外收益”的功能。

虚函数地产 `VirtualfuncTile` 类，它也是 `PropertyTile` 的派生类。它的特有属性包括百分比收租比例 `m_ratio`、`m_buyDecay` 买入减免、收租减免 `m_rentDecay` 等。游戏中使用虚函数卡改变购买和收租的金额，实际上是通过设置这些参数实现的。

上机课、问答格、商店等格子也是通过修改 `landOn()` 实现的。

- **config.h**

定义了游戏全局配置，如每个地块的坐标、名称、类型、颜色组、购买金额、建房金额、收租金额、介绍等信息。

- **dice.h 和 dice.cpp**

定义了骰子类。一次生成并储存两个 1 至 6 之间的随机数。

- **questionbank 和 knowledgebank**

储存了所有问答和知识点事件的信息。将问答和知识点事件都定义为类，并包含了随机抽取函数，在调用时返回一个具体问题或知识点的实例。

- **effectcard.h 和 effectcard.cpp**

定义了效果卡类。包含类型、介绍、和用于商店页面的价格参数等参数。包含了 `createEffectCard()` 和 `randomEffectCardType()` 两个类用于随机生成一个效果卡类的实例。对于迭代器卡，还定义了 `iteratorSubtypeSupports()`，接收一个迭代器卡类型的参数，返回其所能支持的迭代器操作类型。

- **界面文件，包含在 ui 文件夹中**

### 三、小组成员分工情况

- 基础框架搭建：陈侯超、冷子涵、张城玮
- 游戏机制设计：陈侯超
- UI 设计：冷子涵、张城玮
- 测试：陈侯超、冷子涵、张城玮
- 视频制作：张城玮
- 报告写作：冷子涵

### 四、项目总结与反思

首先，作为一个较为完备的 Qt 项目，完成它让我们基本掌握了 Qt 这一实用工具的主要功能，而不仅限于课堂的浅尝辄止；其次，完成这个项目让我们对于 git 和 github 的使用更加熟练了，团队协作和版本控制的素养在新时代的程序设计与开发中是不可或缺的，这让我们成为更好的新时代程序员。最后，这个项目不仅让我们开发者重温了这学期的主要学习内容——基于 c++ 语言的面向对象编程，创作出的产品，也有助于寓教于乐，让其他《程序设计实习》这门课的学习者以喜闻乐见的游戏形式，学习和巩固相关知识。

至于反思，复盘时我们意识到，我们的代码在结构上还是存在一些不清晰的情况，毕竟我们都还是初学者，各自的“行文习惯”也有所不同，所以难免还是写出了一些“屎山代码”；我们的创意，毕竟是三个人一拍脑袋想出来的，路演之后我们也收到了一些对我们的项目的批评建议，我们可能在游戏性上还有进步空间。但无论如何，项目整体无疑具有相当积极的意义，它像一把钥匙，引领我们深入探索程序设计王国，并取得了一些宝藏。